


浙江大學
ZHEJIANG UNIVERSITY

# 计算机组成与设计实验项目报告

cache 仿真器与性能分析

姓名 _____

学号 _____

院所 信息与电子工程学院

2024 年 12 月 20 日

# 目录

**1 项目介绍** **3**

**2 项目任务与要求** **3**

**3 任务 1:trace-driven cache simulator 具体实现** **5**

3.1 cache 初始化 . . . . . 5
3.2 基础功能的 cache 实现 . . . . . 5
3.3 增加可变关联性功能 . . . . . 6
3.4 增加数据写和指令读的统计 . . . . . 7
3.5 增加 split cache 功能 . . . . . 10
3.6 增加 write through 和 write no allocate 策略 . . . . . 10

**4 任务 2: 评估不同的高速缓存体系结构特性的性能** **15**

4.1 spice.trace . . . . . 19
4.2 cc.trace . . . . . 19
4.3 tex.trace . . . . . 19

**5 项目总结** **26**

# 1 项目介绍

实现一个跟踪驱动的缓存模拟器，并使用它来评估不同的缓存体系结构特性的性能。

# 2 项目任务与要求

## 任务 1: 实现 trace-driven cache simulator

高速缓存模拟器将可以根据命令行中给出的参数进行配置，并且必须支持以下功能：

- Total cache size
- Block size
- Unified vs. split I- and D-caches
- Associativity
- Write back vs. write through
- Write allocate vs. write no allocate

模拟器必须跟踪：

- Number of instruction references
- Number of data references
- Number of instruction misses
- Number of data misses
- Number of words fetched from memory
- Number of words copied back to memory

## 任务 2: 评估不同的高速缓存体系结构特性的性能

- characterize the working set size of the three sample traces given.
- evaluate the Impact of Block Size on performance
- evaluate the Impact of Associativity on performance
- evaluate the Impact of Memory Bandwidth on performance

# 3　任务 1:trace-driven cache simulator 具体实现

## 3.1　cache 初始化

先实现 cache 的初始化，命令行参数获取和设置代码框架已经给出，只需要在 cache.c 中实现 init_cache 函数即可。

具体代码实现如下图所示，这里先考虑最简单的情况，即 i-d 统一，直接映射，写回，写分配的 cache。该函数的功能是初始化 cache，根据 cache.c 中已经设置好的静态变量，初始化 c1 结构体中的各个参数并分配内存。同时初始化统计信息的结构体 cache_stat_inst 和 cache_stat_data。

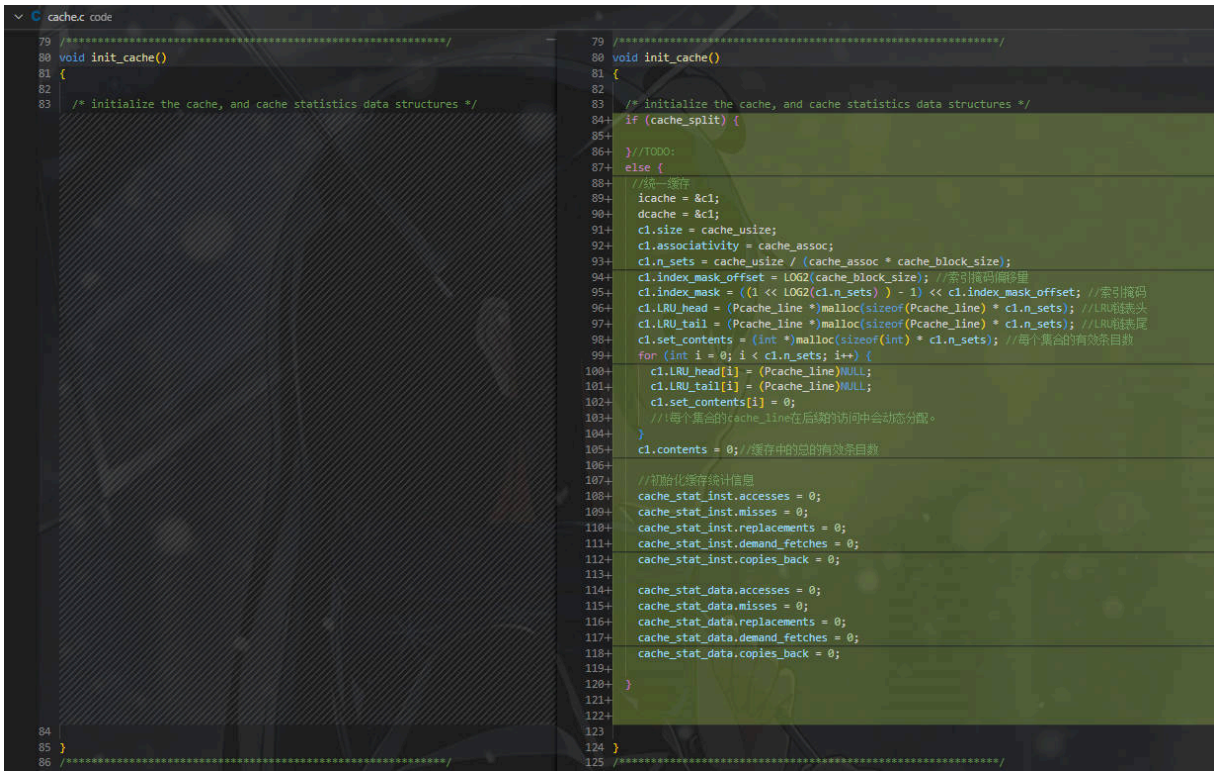



图 1: cache 初始化

## 3.2　基础功能的 cache 实现

接下来实现 perform_access 函数，该函数对 cache 进行仿真，根据访问的地址和类型，更新 cache 的状态。

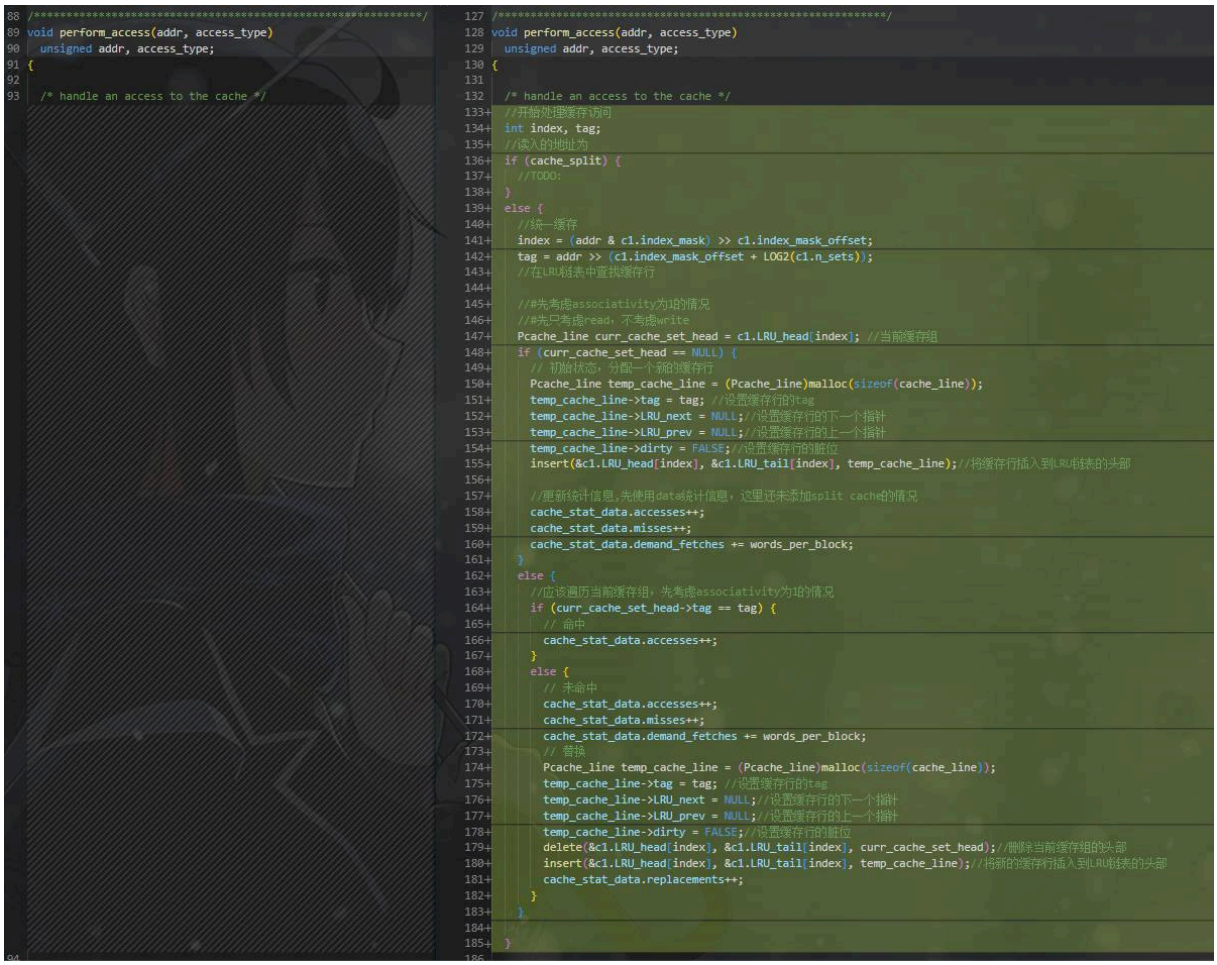
具体代码实现如下图所示：

同样先实现 unify cache 的情况，

先根据 address 计算出 tag 和 index，然后根据 index 找到对应的 cache 组，如果缓存组为空，说明没有命中，需要动态分配一个 cache 块，同时更新统计信息。

否则缓存组不为空，根据双向链表的数据结构，遍历链表，找到对应的 cache 块，(目前 associativity 为 1，即直接映射代码尚未实现)。如果命中，则更新 cache 块的状

态，同时更新统计信息。如果未命中，则根据写策略和写分配策略，更新 cache 块的状态，同时更新统计信息。



```
/************************************************************/
void perform_access(addr, access_type)
  unsigned addr, access_type;
{

  /* handle an access to the cache */
```

```
/************************************************************/
void perform_access(addr, access_type)
  unsigned addr, access_type;
{

  /* handle an access to the cache */
  //开始处理缓存访问
  int index, tag;
  //读入的地址为
  if (cache_split) {
    //TODO:
  }
  else {
    //统一缓存
    index = (addr & c1.index_mask) >> c1.index_mask_offset;
    tag = addr >> (c1.index_mask_offset + LOG2(c1.n_sets));
    //在LRU链表中查找缓存行

    //#先考虑associativity为1的情况
    //#先只考虑read，不考虑write
    Pcache_line curr_cache_set_head = c1.LRU_head[index]; //当前缓存组
    if (curr_cache_set_head == NULL) {
      // 初始状态，分配一个新的缓存行
      Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
      temp_cache_line->tag = tag; //设置缓存行的tag
      temp_cache_line->LRU_next = NULL;//设置缓存行的下一个指针
      temp_cache_line->LRU_prev = NULL;//设置缓存行的上一个指针
      temp_cache_line->dirty = FALSE;//设置缓存行的脏位
      insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line);//将缓存行插入到LRU链表的头部

      //更新统计信息，先使用data统计信息，这里还未添加split cache的情况
      cache_stat_data.accesses++;
      cache_stat_data.misses++;
      cache_stat_data.demand_fetches += words_per_block;
    }
    else {
      //应该遍历当前缓存组，先考虑associativity为1的情况
      if (curr_cache_set_head->tag == tag) {
        // 命中
        cache_stat_data.accesses++;
      }
      else {
        // 未命中
        cache_stat_data.accesses++;
        cache_stat_data.misses++;
        cache_stat_data.demand_fetches += words_per_block;
        // 替换
        Pcache_line temp_cache_line = (Pcache_line)malloc(sizeof(cache_line));
        temp_cache_line->tag = tag; //设置缓存行的tag
        temp_cache_line->LRU_next = NULL;//设置缓存行的下一个指针
        temp_cache_line->LRU_prev = NULL;//设置缓存行的上一个指针
        temp_cache_line->dirty = FALSE;//设置缓存行的脏位
        delete(&c1.LRU_head[index], &c1.LRU_tail[index], curr_cache_set_head);//删除当前缓存组的头部
        insert(&c1.LRU_head[index], &c1.LRU_tail[index], temp_cache_line);//将新的缓存行插入到LRU链表的头部
        cache_stat_data.replacements++;
      }
    }
  }
}
```

图 2: cache 实现

## 3.3　增加可变关联性功能

接下来实现可变关联性的 cache，即 set-associative cache。

如下图 3 所示：首先在初始分配 cache 组的时候，需要加上 cache 结构体中对每组的 cache 块的有效条目数和总的条目数的设置。

如下图 4 所示：增加遍历 cache 组的功能，寻找对应的 cache 块。然后判断是否命中，更新 cache 块的状态和统计信息。如果未命中，还要判断组是否已满，如果未满，则动态分配一个 cache 块，否则根据 LRU 替换策略，选择一个 cache 块替换。然后更新 cache 的统计信息。

这里的 LRU 替换就是将最近使用的 cache 块放在链表的头部，最久未使用的 cache 块放在链表的尾部，每次替换时，将链表尾部的 cache 块替换掉。

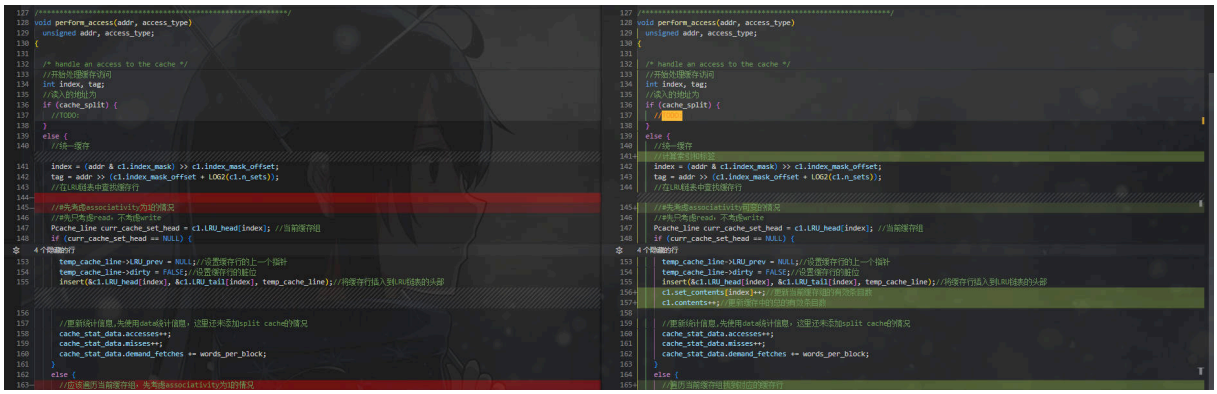


图 3: 可变关联性 cache

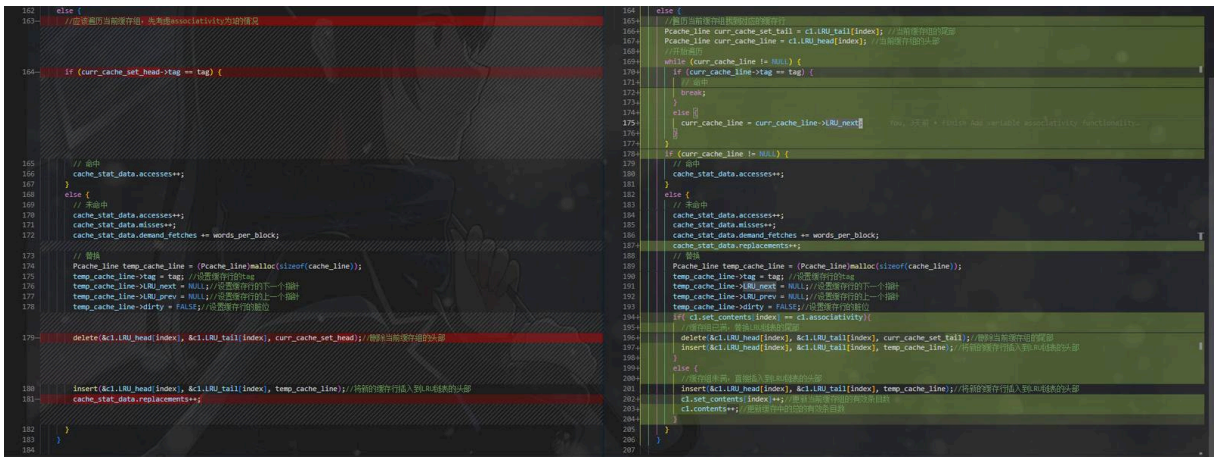


图 4: 可变关联性 cache

## 3.4　增加数据写和指令读的统计

目前的 cache 只能实现对 data 的读取，接下来需要增加对指令的读取和数据写的统计。(目前默认按照 write back 和 write allocate 的策略先进行实现)

如下图 5 所示：首先在分配 cache 块的时候，需要加上对 cache 块的 dirty 标志位的设置。判断是否是数据写，如果是数据写，需要将 dirty 标志位置为 1。在更新统计信息的时候，需要根据访问的类型，更新对应的统计信息。

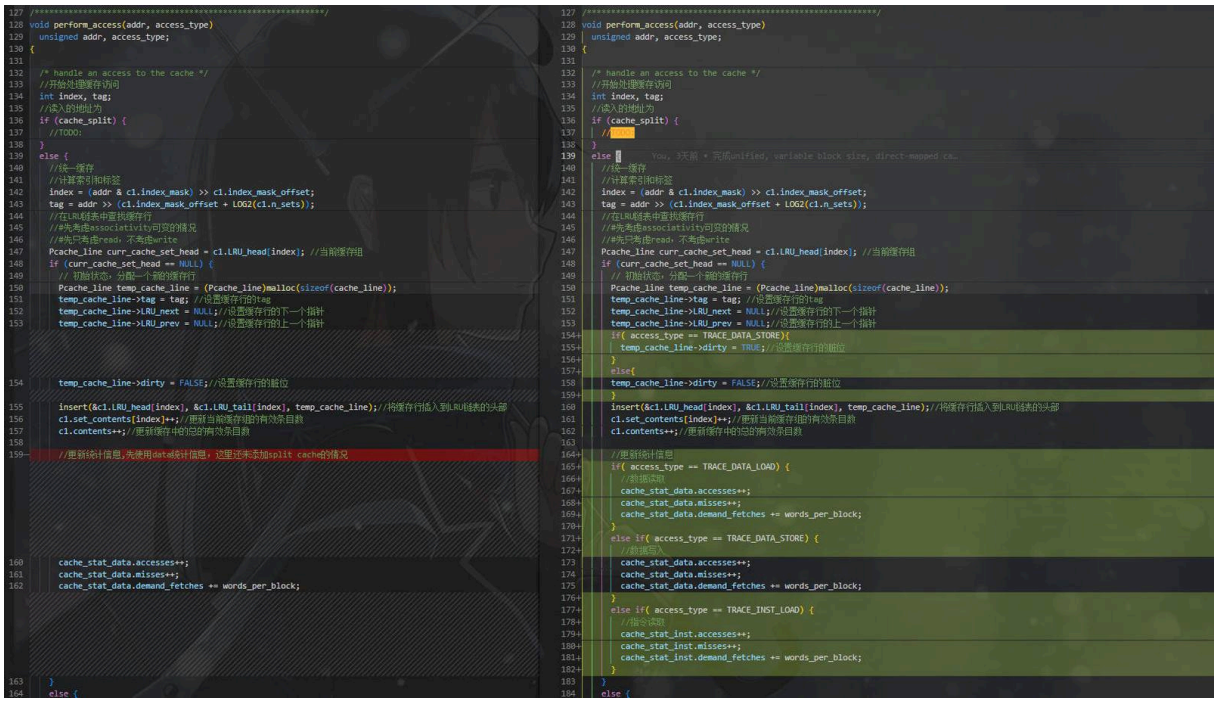


图 5: 增加数据写和指令读的统计

如下图 6，在 cache 命中后，增加了数据写和指令读的统计信息。

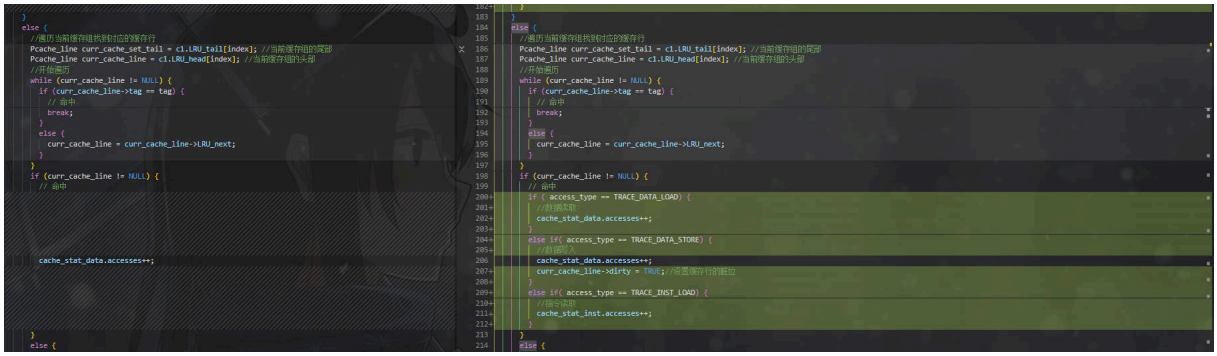


图 6: 增加数据写和指令读的统计

如下图 7，在 cache 未命中后，增加了数据写和指令读的统计信息。并且增加了对 dirty 标志位的判断，如果 dirty 标志位为 1，说明需要将 cache 块的数据写回到内存中，更新 write back 和 replace 的统计信息。同时在替换 cache 块的时候，需要根据访问类型，选择是否设置 dirty 标志位。

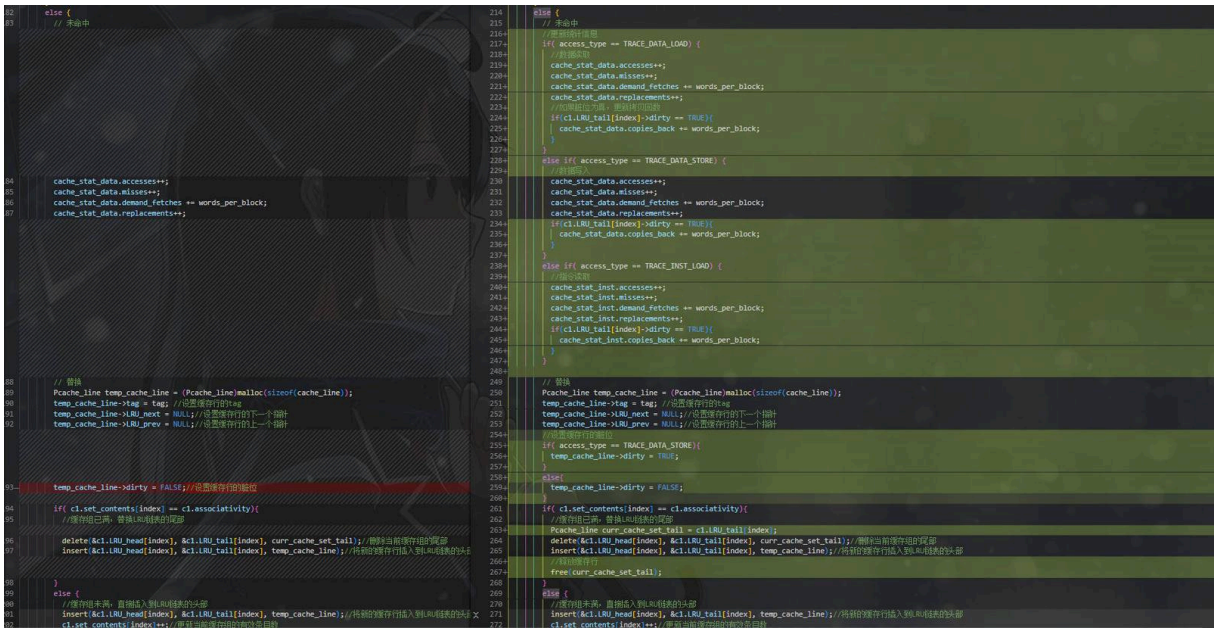


图 7: 增加数据写和指令读的统计

## 增加 flush 功能

在增加写入和读取的统计信息后，需要增加 flush 功能，即在 cache 模拟结束后，将 cache 清空，同时将 dirty 标志位为 1 的 cache 块写回到内存中。并更新统计信息。

如下图 8 所示：flush 函数只需要遍历 cache，将 dirty 标志位为 1 的 cache 块写回到内存中，同时更新统计信息。然后释放分配的内存。



```
214 void flush()
215 {
216
217   /* flush the cache */
218
219 }
```

```
284 void flush()
285 {
286
287   /* flush the cache */
288+  //清空缓存,并将缓存内容写回内存，释放缓存行
289+  if (cache_split) {
290+    //TODO
291+  }
292+  else {
293+    //清空c1缓存
294+    for (int i = 0; i < c1.n_sets; i++) {
295+      Pcache_line curr_cache_line = c1.LRU_tail[i];
296+      while (curr_cache_line != NULL) {
297+        Pcache_line temp= curr_cache_line;
298+        curr_cache_line = curr_cache_line->LRU_prev;
299+        //判断缓存行的脏位，如果是脏的，则写回内存
300+        if(temp->dirty == TRUE){
301+          cache_stat_data.copies_back += words_per_block;
302+        }
303+        //释放缓存行
304+        delete(&c1.LRU_head[i], &c1.LRU_tail[i], temp);
305+        free(temp);
306+      }
307+
308+    }
309+    free(c1.LRU_head);
310+    free(c1.LRU_tail);
311+    free(c1.set_contents);
312+
313+  }
314+
315+
316
317 }
```

图 8: 增加 flush 功能

## BUG 修复

经过测试，发现 replace 和 write back 的统计信息有问题。经过调试，发现了没有更新 LRU 链表和统计信息时候没有判断 LRU 链表是否已满的 bug

如下图 9 所示：在每次访问后，需要更新 LRU 链表，将访问的 cache 块放在链表的头部。在统计信息时，需要判断 LRU 链表是否已满，只要已满，才要进行替换，增加 replace 和 writeback 的统计信息。

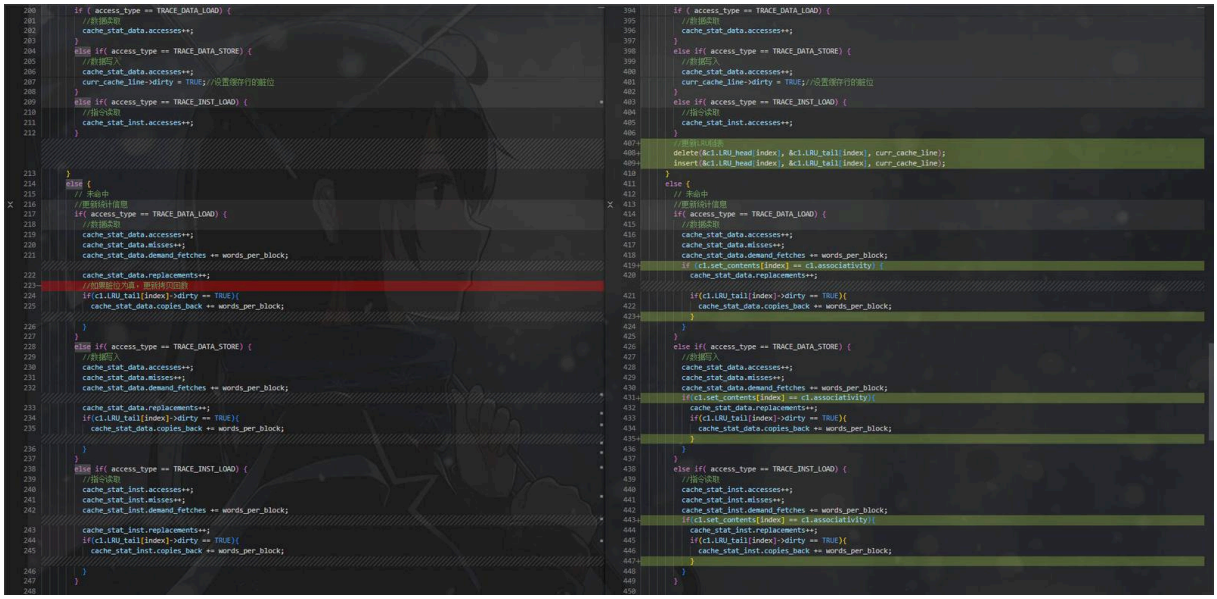


图 9: BUG 修复

## 3.5　增加 split cache 功能

之后增加 split cache 功能，即将 i-d cache 分开，分别对指令和数据分别进行 cache。

这里的实现思路和 unify cache 的实现思路基本一致，只是需要根据访问的类型选择对应的 cache 进行访问即可，其余的逻辑基本一致。实现的思路和前面的 unify cache 的实现思路基本一致，这里不再赘述。详细过程参考源码。

## 3.6　增加 write through 和 write no allocate 策略

### 增加 write through

之后增加 write through 的功能，即写直达策略。写直达策略和写回策略的主要区别在于，写直达策略在写操作时，直接将数据写回到内存中，而写回策略在写操作时，只是将数据写回到 cache 中，只有在 cache 块被替换时，才会将数据写回到内存中。

如下图 10 所示：其实现非常简单，只要在之前代码的基础上，增加对写直达策略的判断即可：

在分配第一个 cache 块的时候，如果为数据写类型访问且是 write through 策略，不需要再设置 dirty 标志位，同时直接增加 write back 的统计信息。

在 cache 命中后，如果为数据写类型访且 write through 策略，需要清除 dirty 标志位设置为 0，同时增加 write back 的统计信息。

在 cache 未命中后，如果为数据写类型访问且 write through 策略，直接增加 write back 的统计信息，且分配的 cache 块也不需要设置 dirty 标志位。

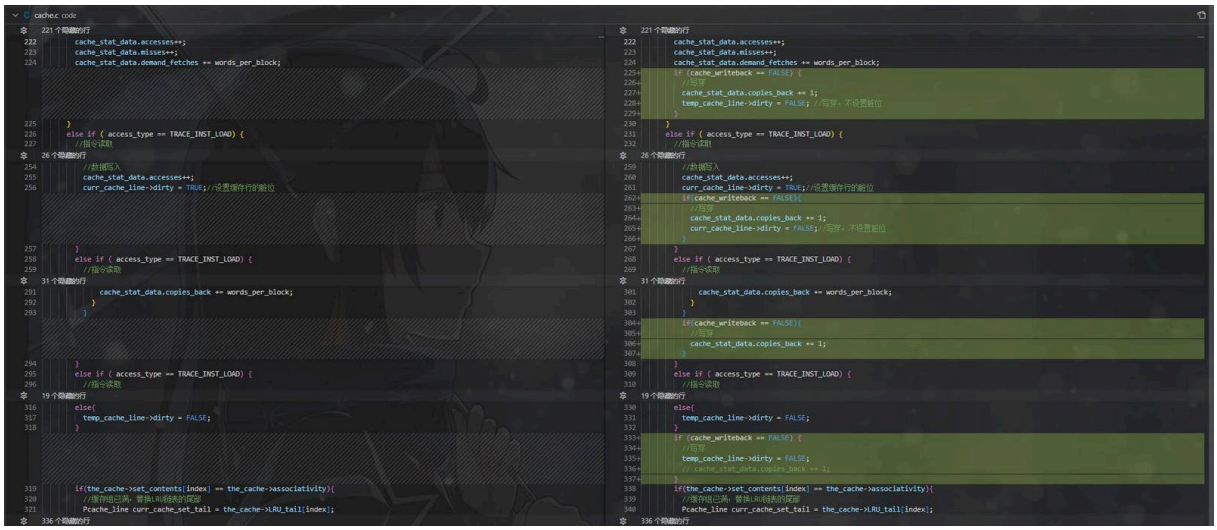


图 10: 增加 write through

## 增加 write no allocate

最后增加 write no allocate 的功能，即写不分配策略。写不分配策略和写分配策略的主要区别在于，写不分配策略在写操作时，如果 cache 未命中，则直接将数据写回到内存中，而不分配 cache 块。而写分配策略在写操作时，如果 cache 未命中，则分配一个 cache 块，并将数据写回到 cache 块中。

如下图 11 所示：首先在初始分配 cache 块的时候，如果为写数据写类型访问且是 write no allocate 策略，则直接跳过分配 cache 块的过程，直接将数据写回到内存中，同时增加 write back 的统计信息。

接着在统计信息时，如果为数据写类型访问且 write no allocate 策略，直接增加 write back，同时不需要 demand fetch 的统计信息。

后续在 cache 未命中时，也是类似的操作，直接将数据写回到内存中，增加 write back，同时不需要 demand fetch，replace 等的统计信息。

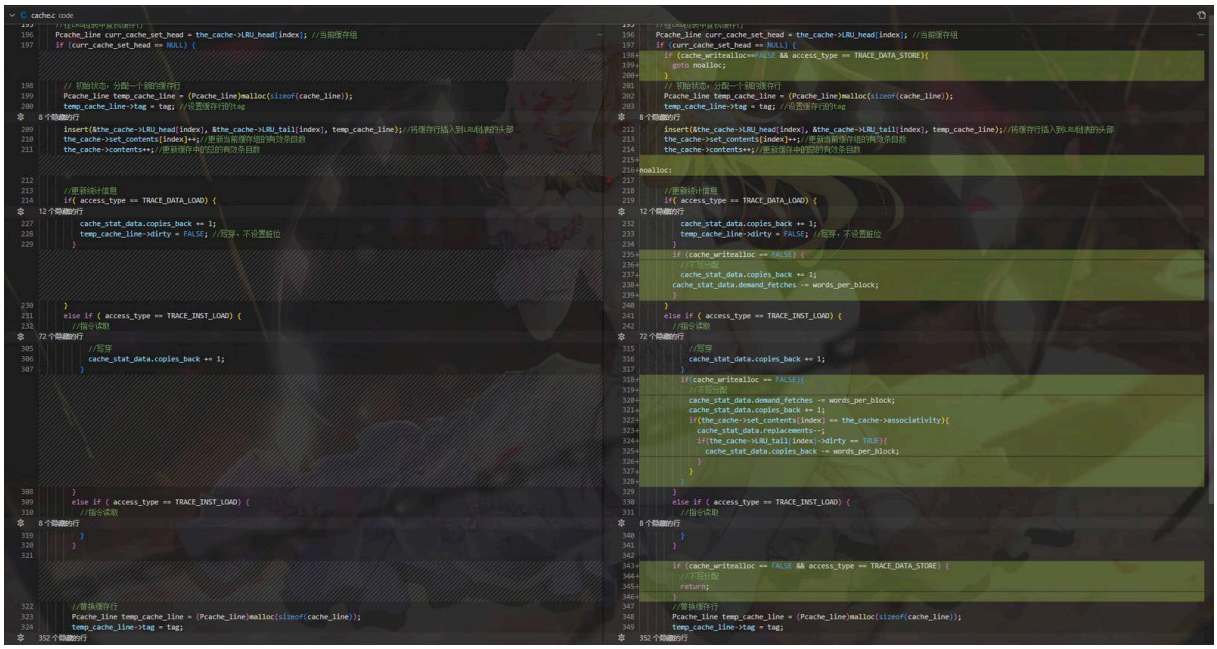


图 11: 增加 write no allocate

**测试结果：**写了一个测试脚本进行测试，测试结果如下：对比表中的实际值，成功通过测试。说明 cache 模拟器功能正确实现。

valid_spice.sh

```
#!/bin/bash

# 定义要运行的命令列表
commands=(
    "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 16 -is 16384 -ds 16384 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 16 -is 32768 -ds 32768 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 16 -is 65536 -ds 65536 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 16 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 32 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -us 8192 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 128 -is 32768 -ds 32768 -a 1 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 4 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 8 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 16 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 128 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 2 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 8 -wb -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 1024 -ds 1024 -a 16 -wb -wa ./ext_traces/spice.trace"

    "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wt -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 32 -is 8192 -ds 8192 -a 1 -wt -wa ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wt -wa ./ext_traces/spice.trace"

    "./code/sim.exe -bs 16 -is 8192 -ds 8192 -a 1 -wa -nw ./ext_traces/spice.trace"
    "./code/sim.exe -bs 32 -is 8192 -ds 8192 -a 1 -wa -nw ./ext_traces/spice.trace"
    "./code/sim.exe -bs 64 -is 8192 -ds 8192 -a 2 -wa -nw ./ext_traces/spice.trace"
)

# 运行命令并提取数据
for cmd in "${commands[@]}"; do
    output=$($cmd)

    # 提取所需的数据
    misses_instr=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "misses" | awk '{print $2}')
    replace_instr=$(echo "$output" | grep -A 5 "INSTRUCTIONS" | grep "replace" | awk '{print $2}')
    misses_data=$(echo "$output" | grep -A 5 "DATA" | grep "misses" | awk '{print $2}')
    replace_data=$(echo "$output" | grep -A 5 "DATA" | grep "replace" | awk '{print $2}')
    demand_fetch=$(echo "$output" | grep "demand fetch" | awk '{print $3}')
    copies_back=$(echo "$output" | grep "copies back" | awk '{print $3}')

    # 打印提取的数据
    echo -e "$misses_instr\t$replace_instr\t$misses_data\t$replace_data\t$demand_fetch\t$copies_back"
done
```

图 12: 测试脚本

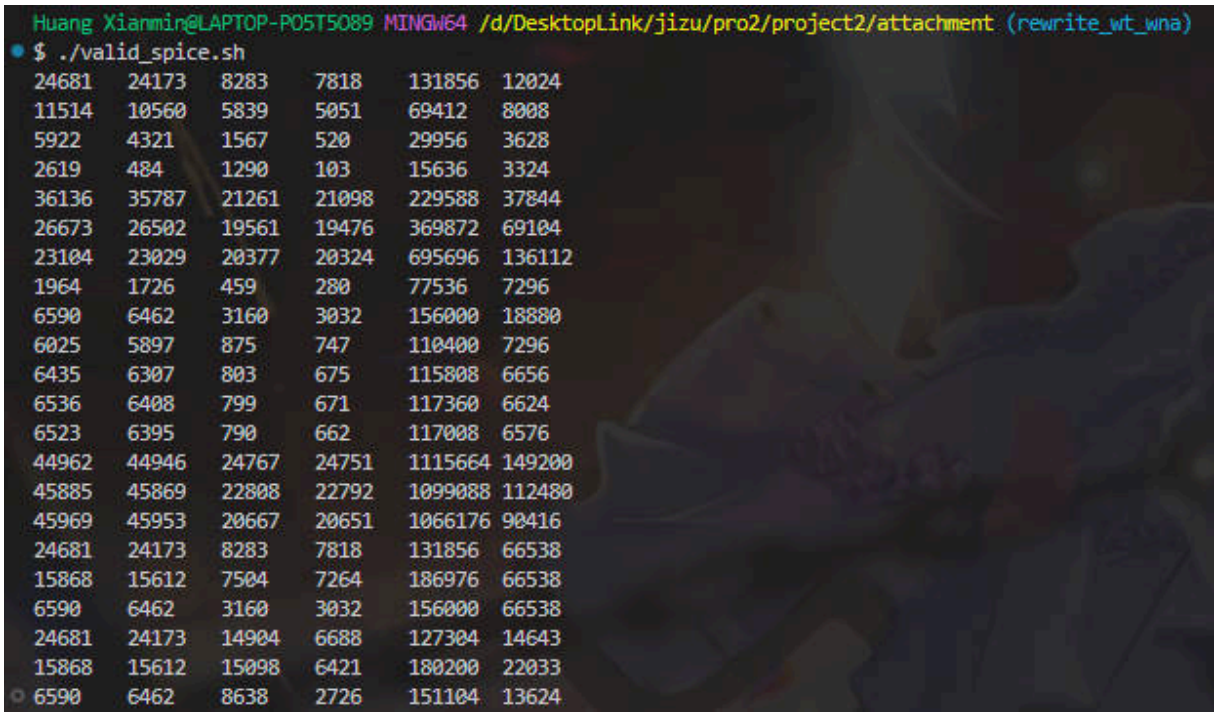



图 13: 测试结果

| | | | | | | Instructions | | Data | | Total | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| CS | I- vs D- | BS | Ass | Write | Alloc | Misses | Repl | Misses | Repl | DF | CB |
| 8 K | Split | 16 | 1 | WB | WA | 24681 | 24173 | 8283 | 7818 | 131856 | 12024 |
| 16 K | Split | 16 | 1 | WB | WA | 11514 | 10560 | 5839 | 5051 | 69412 | 8008 |
| 32 K | Split | 16 | 1 | WB | WA | 5922 | 4321 | 1567 | 520 | 29956 | 3628 |
| 64 K | Split | 16 | 1 | WB | WA | 2619 | 484 | 1290 | 103 | 15636 | 3324 |
| 8 K | Unified | 16 | 1 | WB | WA | 36136 | 35787 | 21261 | 21098 | 229588 | 37844 |
| 8 K | Unified | 32 | 1 | WB | WA | 26673 | 26502 | 19561 | 19476 | 369872 | 69104 |
| 8 K | Unified | 64 | 1 | WB | WA | 23104 | 23029 | 20377 | 20324 | 695696 | 136112 |
| 32 K | Split | 128 | 1 | WB | WA | 1964 | 1726 | 459 | 280 | 77536 | 7296 |
| 8 K | Split | 64 | 2 | WB | WA | 6590 | 6462 | 3160 | 3032 | 156000 | 18880 |
| 8 K | Split | 64 | 4 | WB | WA | 6025 | 5897 | 875 | 747 | 110400 | 7296 |
| 8 K | Split | 64 | 8 | WB | WA | 6435 | 6307 | 803 | 675 | 115808 | 6656 |
| 8 K | Split | 64 | 16 | WB | WA | 6536 | 6408 | 799 | 671 | 117360 | 6624 |
| 8 K | Split | 64 | 128 | WB | WA | 6523 | 6395 | 790 | 662 | 117008 | 6576 |
| 1 K | Split | 64 | 2 | WB | WA | 44962 | 44946 | 24767 | 24751 | 1115664 | 149200 |
| 1 K | Split | 64 | 8 | WB | WA | 45885 | 45869 | 22808 | 22792 | 1099088 | 112480 |
| 1 K | Split | 64 | 16 | WB | WA | 45969 | 45953 | 20667 | 20651 | 1066176 | 90416 |
| 8 K | Split | 16 | 1 | WT | WA | 24681 | 24173 | 8283 | 7818 | 131856 | 66538 |
| 8 K | Split | 32 | 1 | WT | WA | 15868 | 15612 | 7504 | 7264 | 186976 | 66538 |
| 8 K | Split | 64 | 2 | WT | WA | 6590 | 6462 | 3160 | 3032 | 156000 | 66538 |
| 8 K | Split | 16 | 1 | WB | WNA | 24681 | 24173 | 14904 | 6688 | 127304 | 14643 |
| 8 K | Split | 32 | 1 | WB | WNA | 15868 | 15612 | 15098 | 6421 | 180200 | 22033 |
| 8 K | Split | 64 | 2 | WB | WNA | 6590 | 6462 | 8638 | 2726 | 151104 | 13624 |

图 14: 实际值

# 4　任务 2: 评估不同的高速缓存体系结构特性的性能

## Working Set Characterization

**任务要求:**

使用缓存模拟器，将缓存的命中率绘制为缓存大小的函数。从 4 个字节的缓存大小开始，然后增加大小（每次增加 2 倍），直到命中率对缓存大小保持不敏感。使用拆分缓存组织，以便您可以分别描述指令和数据引用的行为（即，每个示例跟踪将有两个图—一个用于指令，一个用于数据）。通过 * 总是”使用完全关联的缓存来排除冲突遗漏的影响。此外，请始终将块大小设置为 4 个字节，使用回写缓存，并使用写分配策略。

写了一个 python 脚本，用于自动测试不同的 cache 大小，得到不同 cache 大小下的命中率。详情见源码。

最终得到的结果如下图所示：

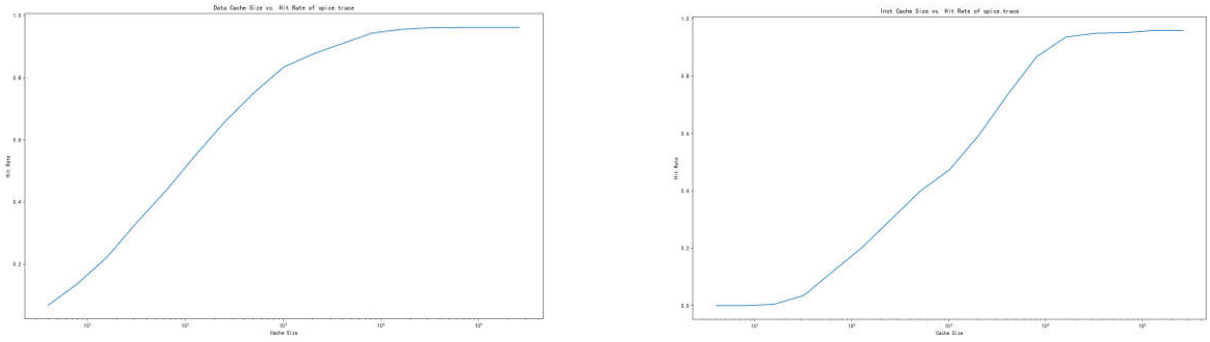


图 15: spice.trace 测试结果

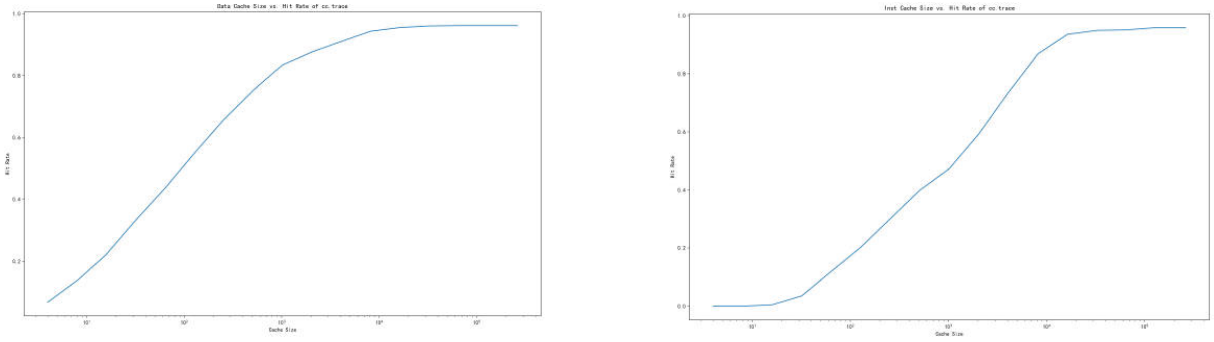


图 16: cc.trace 测试结果

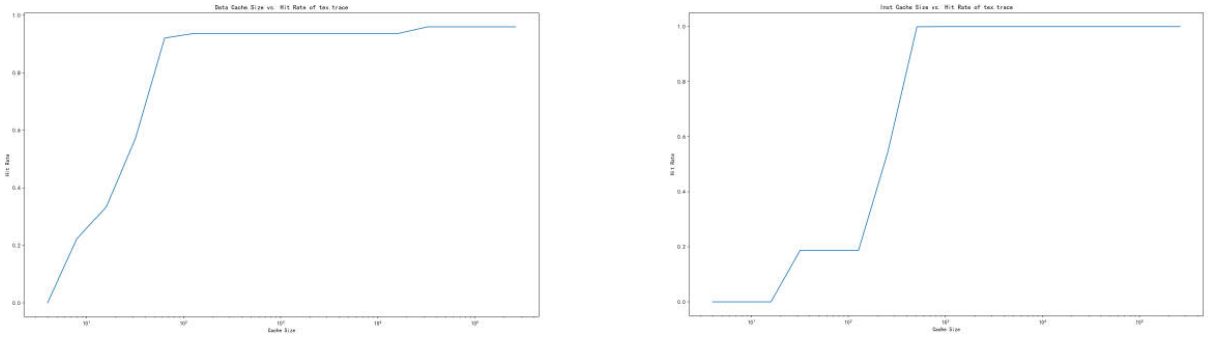


图 17: tex.trace 测试结果

**回答问题:**

1. 解释一下这个实验在做什么，以及它是如何工作的。此外，请解释命中率与缓存大小图中的特性的重要性。

2. 这三个样本跟踪的总指令工作集大小和数据工作集大小是多少？

**问题 1 回答: 实验目的和工作原理**本实验的目的是通过模拟不同缓存大小对命中率的影响，来评估缓存的工作集大小。工作集是指在一定时间内被频繁访问的内存块集合。通过增加缓存大小，我们可以观察到命中率的变化，从而推断出工作集的大小。当缓存大小增加到一定程度，命中率不再显著提高时，表明当前的工作集已经可以被完全包含在缓存中，进一步增加缓存大小不会带来性能上的提升。

实验的工作原理是使用缓存模拟器模拟不同大小的缓存，并记录每次访问时的命中和未命中情况。从 4 字节的缓存大小开始，每次翻倍增加，直到命中率的变化不再显著。通过这种方式，我们可以确定在不同工作负载下，缓存的最佳大小。

**命中率与缓存大小图表的意义**

- **初始快速上升**：在缓存大小较小时，命中率随着缓存大小的增加而迅速上升，这表明工作集大小较小，增加缓存可以显著提高命中率。

- **逐渐平缓**：随着缓存大小的继续增加，命中率的提升速度逐渐减慢，这表明工作集大小正在接近当前缓存大小。

- **趋于平稳**：当缓存大小增加到一定程度后，命中率趋于平稳，这表明工作集已经被完全包含在缓存中，进一步增加缓存大小不会提高命中率。

**问题 2 回答:** 根据上面的分析可知：当缓存大小增加到一定程度后，命中率趋于平稳，这表明工作集已经被完全包含在缓存中，进一步增加缓存大小不会提高命中率。因此，当命中率趋于平稳时，工作集的大小即为当前缓存大小。因此，我们可以通过观察命中率与缓存大小图表的特性来推断工作集的大小。

从上面的三张图片可以看出，spice.trace 的数据工作集大小约为 $2^14Bytes = 16KB$，指令工作集大小也约为 $2^14Bytes = 16KB$

cc.trace 的数据工作集大小约为 $2^14 = 3Bytes = 16 == 8KB$ ，指令工作集大小约为 $2^14Bytes = 16KB$

tex.trace 的数据工作集大小约为 $2^6Bytes = 128B$ ，指令工作集大小约为 $2^9Bytes = 512B$

## Impact of Block Size

**任务要求:**

将缓存模拟器设置为拆分的 i 缓存组织，每个大小为 8k 字节。将关联性设置为 2，使用回写缓存，并使用写分配策略。将高速缓存的命中率绘制为块大小的函数。在 4 字节和 4k 字节之间改变块大小，幂为 2。对这三个轨迹执行此操作，并为指令和数据引用创建一个单独的绘图。

使用 python 脚本，自动测试不同的 block size，得到不同 block size 下的命中率并绘制图表。详情见源码。最终得到的结果如下图所示：

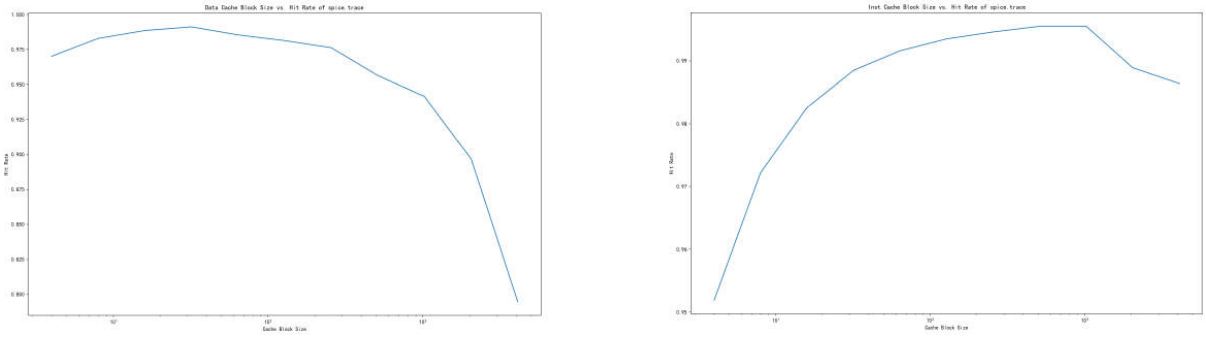


图 18: spice.trace 测试结果

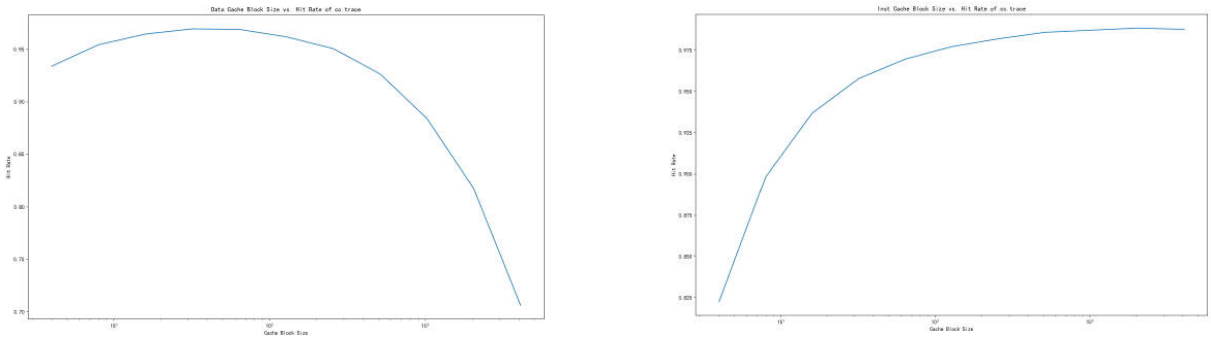


图 19: cc.trace 测试结果

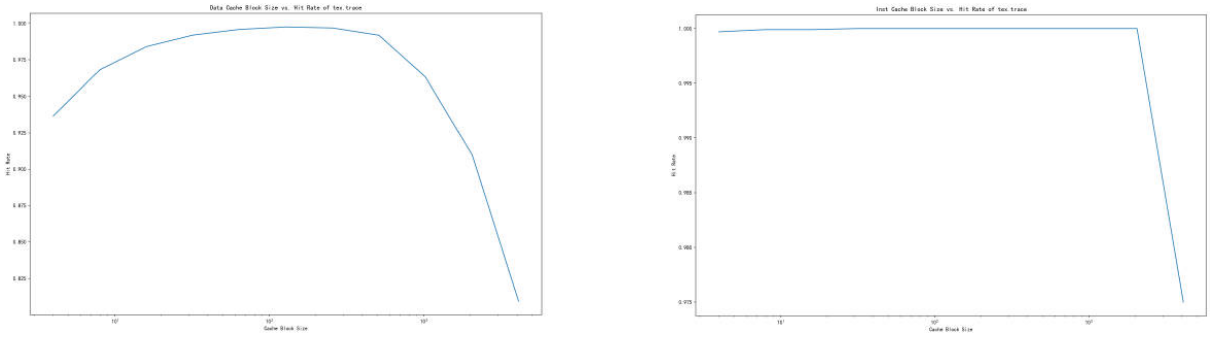


图 20: tex.trace 测试结果

**回答问题:**

1. 解释命中率与块大小图的形状。特别是，解释空间局部性与这条曲线的形状的相关性。

2. 每个跟踪的最佳块大小是多少（分别考虑指令和数据引用）?

3. 指令和数据引用的最佳块大小是否不同？这告诉你关于指令引用和数据引用的性质是什么？

**问题 1 回答:**

- 初始上升阶段：当缓存块大小从较小的值开始增加时，命中率迅速上升。这是因为较小的块大小能够更精细地映射内存区域，使得缓存能够更好地捕捉到程序的空间局部性。如果程序访问了一块内存区域，那么相邻的内存区域很可能也会被访问，小的块大小使得这些相邻区域更有可能被一起加载到缓存中。

- 平台期：随着块大小的进一步增加，命中率的增长速度开始放缓，进入一个平台期。在这个阶段，缓存块大小已经足够大，能够覆盖程序的大部分空间局部性需求。即使块大小继续增加，对命中率的提升也不再显著，因为缓存已经能够有效地利用空间局部性。

- 下降阶段：当块大小增加到一定程度后，命中率开始下降。这是因为过大的块大小可能会导致缓存中存储的块包含许多不相关的内存区域，这些区域可能不会被程序访问，从而导致缓存空间的浪费。此外，大的块大小可能会覆盖多个可能被独立访问的内存区域，增加了缓存冲突的可能性，降低了缓存的效率。

- 曲线的拐点：曲线的拐点通常出现在块大小足以覆盖程序的典型空间局部性范围时。在这个点之前，增加块大小可以提高命中率，因为更大的块可以包含更多的局部性区域。在这个点之后，增加块大小反而可能导致命中率下降，因为块内包含的不相关内存区域增多，减少了缓存的有效性。

分析六张图，他们都大致呈现了上述的特性，但是曲线的形状和拐点位置不同，说明不同程序的空间局部性不同，对块大小的需求也不同，所以导致了不同的曲线形状。

**问题 2 回答:** 通过分析每个轨迹的命中率与块大小的关系图，我们可以确定每个轨迹的最佳块大小，即在该块大小下，缓存命中率达到最高或接近最高。

## 4.1 spice.trace

- **指令引用：**从图中可以看出，当块大小达到 1024 字节时，命中率接近最高，之后随着块大小的增加，命中率略有下降。
- **数据引用：**数据引用的命中率在块大小为 32 字节时达到最高，之后随着块大小的增加，命中率开始下降。

## 4.2 cc.trace

- **指令引用：**对于指令引用，块大小为 2048 字节时，命中率最高
- **数据引用：**数据引用的命中率在块大小为 32 字节时达到最高, 之后随着块大小的增加，命中率略有下降。

## 4.3 tex.trace

- **指令引用：**在块大小为 4 字节到 2048 字节之间，命中率基本保持稳定，没有明显的变化，但在块大小为 4096 字节时，命中率开始下降。
- **数据引用：**数据引用的命中率在块大小为 128 字节时达到最高，之后随着块大小的增加，命中率开始下降。

**问题 3 回答:**

通过分析每个轨迹的命中率与块大小的关系图，我们可以确定指令引用和数据引用的最佳块大小。结果显示，指令引用和数据引用的最佳块大小确实不同。

这些结果表明，指令引用和数据引用的性质存在显著差异。指令引用通常具有较强的空间局部性，因为程序的指令通常是顺序执行的，因此较大的块大小可以更好地利用这种局部性。而数据引用则可能更加分散，具有较弱的空间局部性，因此较小的块大小更适合数据引用，以减少缓存空间的浪费和缓存冲突的可能性。

## Impact of Associativity

**任务要求:**

将缓存模拟器设置为拆分的缓存组织，每个大小为 8k 字节。将块大小设置为 128 字节，使用回写缓存，并使用写分配策略。绘制高速缓存的命中率作为关联性的函数。改变 1 和 64 之间的结合性，幂次为 2。对这三个轨迹执行此操作，并为指令和数据引用创建一个单独的绘图。

使用 python 脚本, 自动测试不同的 associativity，得到不同 associativity 下的命中率并绘制图表。详情见源码。

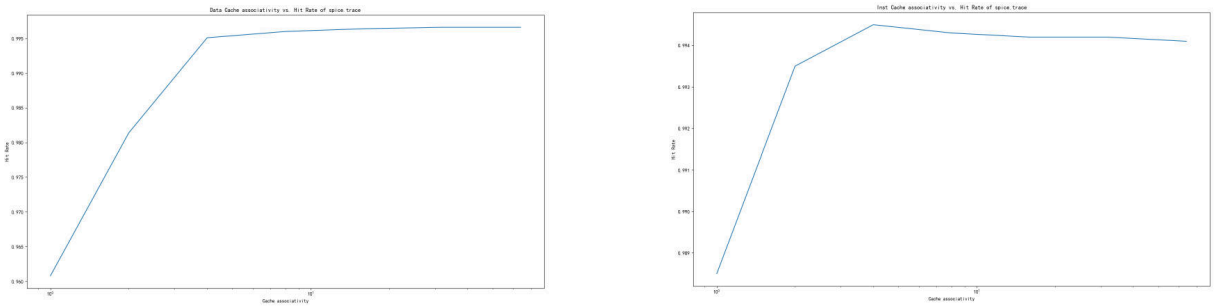


图 21: spice.trace 测试结果

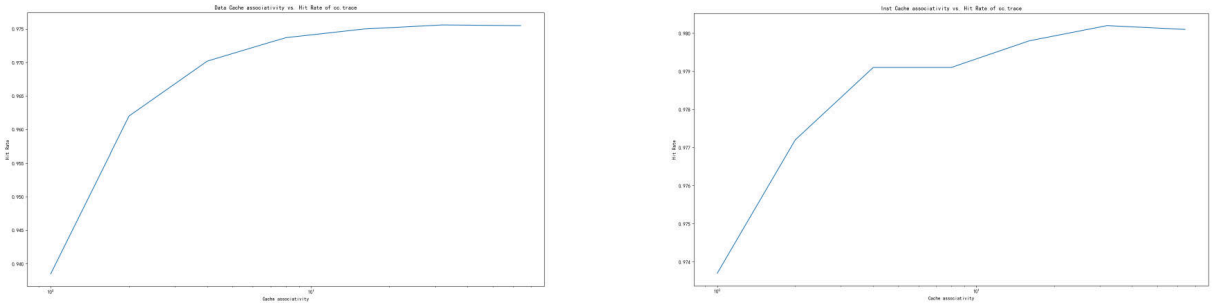


图 22: cc.trace 测试结果

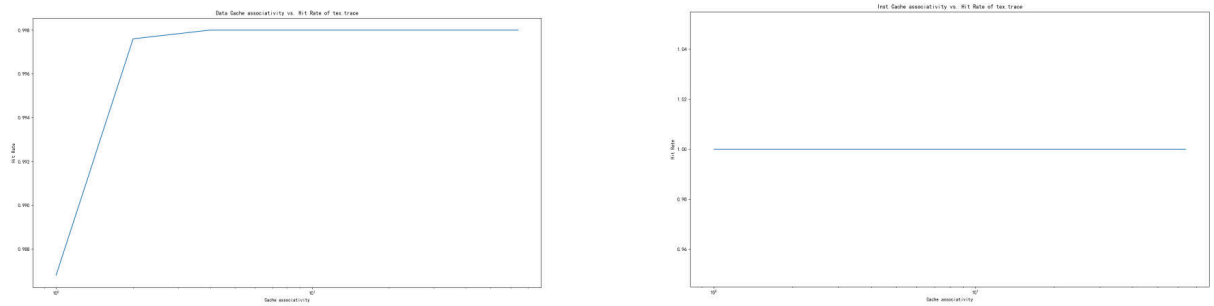


图 23: tex.trace 测试结果

**回答问题:**

1. 解释为什么 hit rate vs. associativity 的图表具有这样的形状。
2. 指令和数据的图之间是否有区别？这告诉了你什么，关于 associativity 对指令和数据引用的影响的差异？

**问题 1 回答:**

**命中率与关联度关系图的形状解释**

命中率与关联度的关系图展示了缓存命中率随关联度变化的趋势。这种趋势的形状与缓存的组织结构和程序的内存访问模式密切相关。

**初始上升阶段**

subsubsection* 在关联度较低时，命中率随着关联度的增加而迅速上升。这是因为较低的关联度意味着缓存中的块冲突较多，增加关联度可以减少这些冲突，从而提高命中率。

**平台期**

当关联度增加到一定程度后，命中率的提升速度开始放缓，进入一个平台期。在这个阶段，缓存已经能够较好地减少冲突，进一步增加关联度对命中率的提升效果有限。

**某些情况出现下降阶段**

在某些情况下，当关联度继续增加超过某个阈值后，命中率可能会略有下降。这可能是由于缓存结构的复杂性增加，导致缓存管理的开销增大，或者在某些极端情况下，过大的关联度可能导致缓存的利用率降低。

**其它特殊情况：**

如 tex.trace 的 icache 图，命中率随着关联度几乎保持在 100%，说明该程序的指令引用具有很好的空间局部性，无论关联度如何，都能被缓存很好地覆盖。

**结论**

缓存的关联度对命中率有显著影响。合理选择关联度对于优化缓存性能至关重要。在实际应用中，需要根据程序的特性和访问模式来调整关联度，以达到最佳的缓存效率。

**问题 2 回答:**

### 差异分析

- **上升阶段**：对于指令和数据引用，命中率随着关联度的增加而上升，但上升的速率和开始平稳的点可能不同。
- **平台期**：在达到一定关联度后，命中率的提升趋于平缓，这个点对于指令和数据引用可能有所不同。
- **下降阶段**：在某些情况下，过高的关联度可能导致命中率略有下降，这在指令和数据引用中的表现可能不一致。

### 差异意义

关联度对指令和数据引用的影响差异反映了两者在缓存行为上的不同：

- **指令引用**：由于指令访问通常具有较好的空间局部性，适当的关联度可以显著提高命中率。
- **数据引用**：数据访问模式可能更加随机，因此关联度对命中率的影响可能不如指令引用明显。

### 结论

关联度对指令和数据引用的命中率有不同的影响。指令引用可能更受益于较高的关联度，而数据引用在达到一定关联度后，命中率的提升可能有限。这表明在设计缓存时，需要考虑不同类型内存访问的特性，以优化缓存性能。

## Memory Bandwidth

**任务要求 1:**

使用缓存模拟器，比较通写缓存和回写缓存生成的总内存流量。使用拆分的高速缓存组织。模拟一些合理的缓存大小（如 8K 字节和 16K 字节）、合理的块大小（如 64 和 128 字节）以及合理的关联性（如 2 和 4）。现在，请使用无写分配策略。总共运行 4 或 5 个不同的模拟。

**回答问题:**

1. 哪个缓存具有较小的内存流量，通写缓存或回写缓存？为什么

2. 在什么情况下，你对上面的问题的回答会翻转吗？解释。

使用 python 脚本, 自动测试不同的 cache size, block size, associativity, write policy，得到不同参数下的内存流量并绘制图表。得到如下结果：

Memory Traffic Simulation Results of spice.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 151104 | 13624 |
| 128 | 8192 | 8192 | 2 | wb | nw | 280768 | 32287 |
| 64 | 16384 | 16384 | 2 | wb | nw | 57008 | 8252 |
| 128 | 16384 | 16384 | 2 | wb | nw | 93632 | 9880 |
| 64 | 8192 | 8192 | 4 | wb | nw | 107296 | 9219 |
| 128 | 8192 | 8192 | 4 | wb | nw | 162240 | 13733 |
| 64 | 16384 | 16384 | 4 | wb | nw | 38064 | 7681 |
| 128 | 16384 | 16384 | 4 | wb | nw | 64320 | 9668 |

Memory Traffic Simulation Results of spice.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wt | nw | 151104 | 72322 |
| 128 | 8192 | 8192 | 2 | wt | nw | 280768 | 72777 |
| 64 | 16384 | 16384 | 2 | wt | nw | 57008 | 71430 |
| 128 | 16384 | 16384 | 2 | wt | nw | 93632 | 71490 |
| 64 | 8192 | 8192 | 4 | wt | nw | 107296 | 71453 |
| 128 | 8192 | 8192 | 4 | wt | nw | 162240 | 71599 |
| 64 | 16384 | 16384 | 4 | wt | nw | 38064 | 71067 |
| 128 | 16384 | 16384 | 4 | wt | nw | 64320 | 71086 |

图 24: spice.trace 测试结果

Memory Traffic Simulation Results of cc.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 474256 | 39426 |
| 128 | 8192 | 8192 | 2 | wb | nw | 824192 | 78872 |
| 64 | 16384 | 16384 | 2 | wb | nw | 252096 | 24048 |
| 128 | 16384 | 16384 | 2 | wb | nw | 440768 | 40538 |
| 64 | 8192 | 8192 | 4 | wb | nw | 423104 | 32102 |
| 128 | 8192 | 8192 | 4 | wb | nw | 716736 | 59292 |
| 64 | 16384 | 16384 | 4 | wb | nw | 205120 | 19588 |
| 128 | 16384 | 16384 | 4 | wb | nw | 353632 | 31103 |

Memory Traffic Simulation Results of cc.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wt | nw | 474256 | 94312 |
| 128 | 8192 | 8192 | 2 | wt | nw | 824192 | 93582 |
| 64 | 16384 | 16384 | 2 | wt | nw | 252096 | 91382 |
| 128 | 16384 | 16384 | 2 | wt | nw | 440768 | 90416 |
| 64 | 8192 | 8192 | 4 | wt | nw | 423104 | 92940 |
| 128 | 8192 | 8192 | 4 | wt | nw | 716736 | 92370 |
| 64 | 16384 | 16384 | 4 | wt | nw | 205120 | 90954 |
| 128 | 16384 | 16384 | 4 | wt | nw | 353632 | 90069 |

图 25: cc.trace 测试结果

Memory Traffic Simulation Results of tex.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 2880 | 29903 |
| 128 | 8192 | 8192 | 2 | wb | nw | 3488 | 29935 |
| 64 | 16384 | 16384 | 2 | wb | nw | 2464 | 29909 |
| 128 | 16384 | 16384 | 2 | wb | nw | 2656 | 29957 |
| 64 | 8192 | 8192 | 4 | wb | nw | 3312 | 29903 |
| 128 | 8192 | 8192 | 4 | wb | nw | 4064 | 29935 |
| 64 | 16384 | 16384 | 4 | wb | nw | 2464 | 29909 |
| 128 | 16384 | 16384 | 4 | wb | nw | 2656 | 29957 |

Memory Traffic Simulation Results of tex.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wt | nw | 2880 | 134384 |
| 128 | 8192 | 8192 | 2 | wt | nw | 3488 | 134384 |
| 64 | 16384 | 16384 | 2 | wt | nw | 2464 | 134374 |
| 128 | 16384 | 16384 | 2 | wt | nw | 2656 | 134374 |
| 64 | 8192 | 8192 | 4 | wt | nw | 3312 | 134384 |
| 128 | 8192 | 8192 | 4 | wt | nw | 4064 | 134384 |
| 64 | 16384 | 16384 | 4 | wt | nw | 2464 | 134374 |
| 128 | 16384 | 16384 | 4 | wt | nw | 2656 | 134374 |

图 26: tex.trace 测试结果

**问题 1 回答:** 对比图中的 damand_fetch 和 copies_back 可以发现 write back cache 具有较小的内存流量。这是因为回写缓存策略下，只有当数据被修改时，才会将数据写回主存，而通写缓存策略下，每次数据修改都会立即写回主存，导致更多的写操作，从而增加内存流量。

**问题 2 回答:** 在以下情况下，通写缓存的内存流量可能会比回写缓存小：

- **写操作频繁且数据不经常被重写：**如果程序中写操作非常频繁，并且写入的数据很少被重写，那么通写缓存可能会比回写缓存更有效。因为在回写缓存中，每次写操作都需要将数据标记为“脏”，并在替换时写回主存，这会导致额外的内存流量。而在通写缓存中，每次写操作都会立即写回主存，避免了多次写回相同数据的情况。
- **数据一致性要求高：**在某些系统中，数据一致性要求很高，需要确保每次写操作后数据立即更新到主存。在这种情况下，通写缓存可以减少由于数据一致性问题而导致的额外内存流量。
- **缓存替换频繁：**如果缓存替换非常频繁，回写缓存可能会导致大量的写回操作，从而增加内存流量。而通写缓存每次写操作都立即写回主存，避免了替换时的写回操作。

总的来说，当写操作频繁且数据不经常被重写，或者系统对数据一致性要求高时，通写缓存的内存流量可能会比回写缓存小。

**任务要求 2:**

现在使用相同的参数，但修复策略以进行回写。执行相同的实验，但这次比较写分配和写不分配策略。

**回答问题:**

1. 哪个缓存具有较小的内存流量、写分配缓存或无写分配缓存？为什么
2. 在什么情况下，你对上面的问题的回答会翻转吗？解释。

使用 python 脚本, 自动测试不同的 cache size, block size, associativity, write policy，得到不同参数下的内存流量并绘制图表。得到如下结果：

Memory Traffic Simulation Results of spice.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | wa | 156000 | 18880 |
| 128 | 8192 | 8192 | 2 | wb | wa | 291584 | 36256 |
| 64 | 16384 | 16384 | 2 | wb | wa | 59856 | 6208 |
| 128 | 16384 | 16384 | 2 | wb | wa | 99008 | 9088 |
| 64 | 8192 | 8192 | 4 | wb | wa | 110400 | 7296 |
| 128 | 8192 | 8192 | 4 | wb | wa | 171136 | 14592 |
| 64 | 16384 | 16384 | 4 | wb | wa | 39728 | 5280 |
| 128 | 16384 | 16384 | 4 | wb | wa | 66304 | 7200 |

Memory Traffic Simulation Results of spice.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 151104 | 13624 |
| 128 | 8192 | 8192 | 2 | wb | nw | 280768 | 32287 |
| 64 | 16384 | 16384 | 2 | wb | nw | 57008 | 8252 |
| 128 | 16384 | 16384 | 2 | wb | nw | 93632 | 9880 |
| 64 | 8192 | 8192 | 4 | wb | nw | 107296 | 9219 |
| 128 | 8192 | 8192 | 4 | wb | nw | 162240 | 13733 |
| 64 | 16384 | 16384 | 4 | wb | nw | 38064 | 7681 |
| 128 | 16384 | 16384 | 4 | wb | nw | 64320 | 9668 |

图 27: spice.trace 测试结果

Memory Traffic Simulation Results of cc.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | wa | 488976 | 41840 |
| 128 | 8192 | 8192 | 2 | wb | wa | 848832 | 91744 |
| 64 | 16384 | 16384 | 2 | wb | wa | 261248 | 23584 |
| 128 | 16384 | 16384 | 2 | wb | wa | 455840 | 46048 |
| 64 | 8192 | 8192 | 4 | wb | wa | 435408 | 33216 |
| 128 | 8192 | 8192 | 4 | wb | wa | 737632 | 68384 |
| 64 | 16384 | 16384 | 4 | wb | wa | 212176 | 18208 |
| 128 | 16384 | 16384 | 4 | wb | wa | 363392 | 32512 |

Memory Traffic Simulation Results of cc.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 474256 | 39426 |
| 128 | 8192 | 8192 | 2 | wb | nw | 824192 | 78872 |
| 64 | 16384 | 16384 | 2 | wb | nw | 252096 | 24048 |
| 128 | 16384 | 16384 | 2 | wb | nw | 440768 | 40538 |
| 64 | 8192 | 8192 | 4 | wb | nw | 423104 | 32102 |
| 128 | 8192 | 8192 | 4 | wb | nw | 716736 | 59292 |
| 64 | 16384 | 16384 | 4 | wb | nw | 205120 | 19588 |
| 128 | 16384 | 16384 | 4 | wb | nw | 353632 | 31103 |

图 28: cc.trace 测试结果

Memory Traffic Simulation Results of tex.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | wa | 15696 | 7664 |
| 128 | 8192 | 8192 | 2 | wb | wa | 18528 | 8608 |
| 64 | 16384 | 16384 | 2 | wb | wa | 11696 | 7616 |
| 128 | 16384 | 16384 | 2 | wb | wa | 13696 | 8256 |
| 64 | 8192 | 8192 | 4 | wb | wa | 15408 | 7568 |
| 128 | 8192 | 8192 | 4 | wb | wa | 15648 | 7648 |
| 64 | 16384 | 16384 | 4 | wb | wa | 10480 | 7568 |
| 128 | 16384 | 16384 | 4 | wb | wa | 11200 | 7648 |

Memory Traffic Simulation Results of tex.trace

| bs | is | ds | a | write_policy | allocation_policy | demand_fetch | copies_back |
|---|---|---|---|---|---|---|---|
| 64 | 8192 | 8192 | 2 | wb | nw | 2880 | 29903 |
| 128 | 8192 | 8192 | 2 | wb | nw | 3488 | 29935 |
| 64 | 16384 | 16384 | 2 | wb | nw | 2464 | 29909 |
| 128 | 16384 | 16384 | 2 | wb | nw | 2656 | 29957 |
| 64 | 8192 | 8192 | 4 | wb | nw | 3312 | 29903 |
| 128 | 8192 | 8192 | 4 | wb | nw | 4064 | 29935 |
| 64 | 16384 | 16384 | 4 | wb | nw | 2464 | 29909 |
| 128 | 16384 | 16384 | 4 | wb | nw | 2656 | 29957 |

图 29: tex.trace 测试结果

**问题 1 回答:** 观察上面的图表可以发现：对于 spice.trace 和 tex.spice 测试结果，写不分配缓存具有较小的内存流量；对于 tex.trace 测试结果，写分配缓存具有较小的内存流量。通常情况下，写不分配策略可能会产生较小的内存流量。原因在于写不分配策略在写操作发生时不会将新的缓存行加载到缓存中，这可以减少从主存取数据的次数，从而减少内存流量。然而，写分配策略会在写操作时加载新的缓存行，这可能会导致更多的缓存替换和对主存的写操作，从而增加内存流量。但是，这种差异也取决于具体的缓存大小、块大小和程序的访问模式。

**问题 2 回答:** 在程序的空间局部性较高，写操作频繁且数据经常被重写的情况下，写分配缓存可能会产生较小的内存流量。因为写分配缓存会在写操作时加载新的缓存行，并在之后的写操作中重复使用这些缓存行，从而减少了从主存取数据的次数，减少了内存流量。

# 5 项目总结

本项目通过实现一个基于追踪的缓存模拟器，深入评估了不同缓存结构特性对性能的影响。在项目过程中，我们不仅加深了对缓存工作原理的理解，而且通过实践提高了解决复杂问题的能力。

## 主要成果

- 成功实现了一个功能全面的缓存模拟器，支持多种缓存配置，包括缓存大小、块大小、关联度、写策略和分配策略。
- 通过模拟不同的缓存配置，我们评估了工作集大小、块大小、关联度和内存带宽对缓存性能的影响。
- 对比了写直达和写回策略在不同缓存配置下的内存流量，为缓存设计提供了实证数据支持。
- 分析了写分配和写不分配策略在特定条件下的性能差异，进一步优化了缓存策略的选择。

## 经验与反思

- 项目过程中，我们遇到了一些技术挑战，如模拟器的稳定性和性能优化，通过对代码的不断调试，最终克服了这些挑战。
- 实践中我们发现，缓存设计需要综合考虑程序特性和硬件限制，没有一种配置能够适用于所有场景。
- 通过项目，我们更加认识到理论与实践相结合的重要性，以及持续学习和适应新技术的必要性。